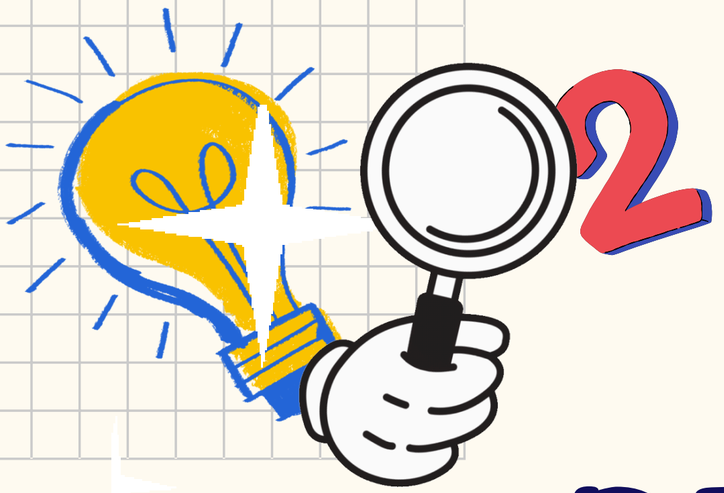
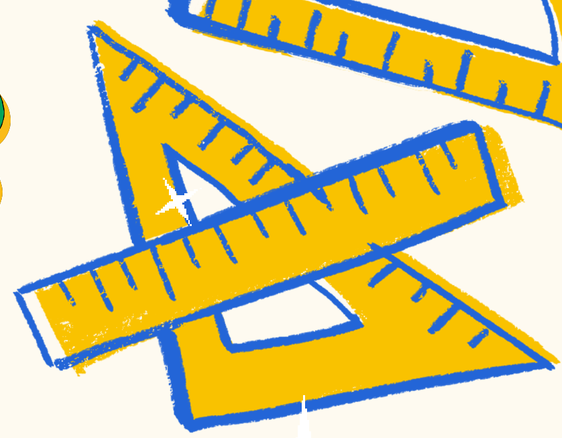
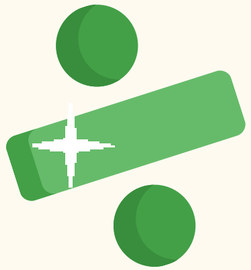






COURSE REGISTRATION SYSTEM





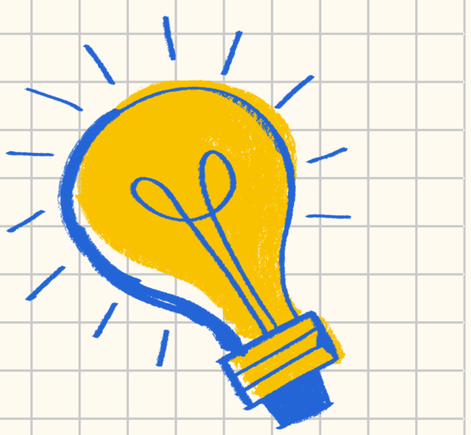
Student course management system















OUR TEAM

Yigitalieve Mubina

20253721

Rahmonova Mohinur

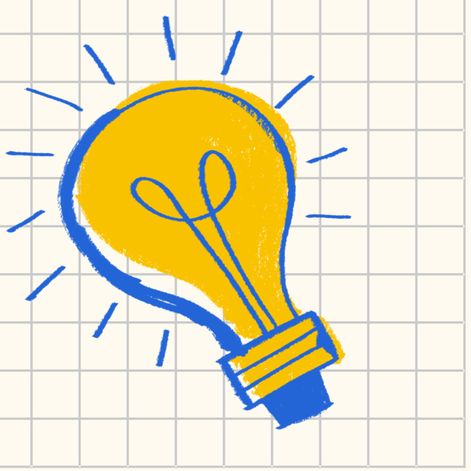
20253719



SYSTEM OVERVIEW

This project is a functional, command-line system designed to simulate the core academic and administrative functions of a university. Its main purpose is to manage student registration, the course catalog, and enrollment records in a secure and efficient terminal environment.

The system is structured with specialized Service Managers to handle all data logic: one for User Authentication (Admins and Students), one for Course Management (adding, updating, and removing courses), and a third for Enrollment Processing (handling student sign-ups, drops, and capacity limits).



GOALS

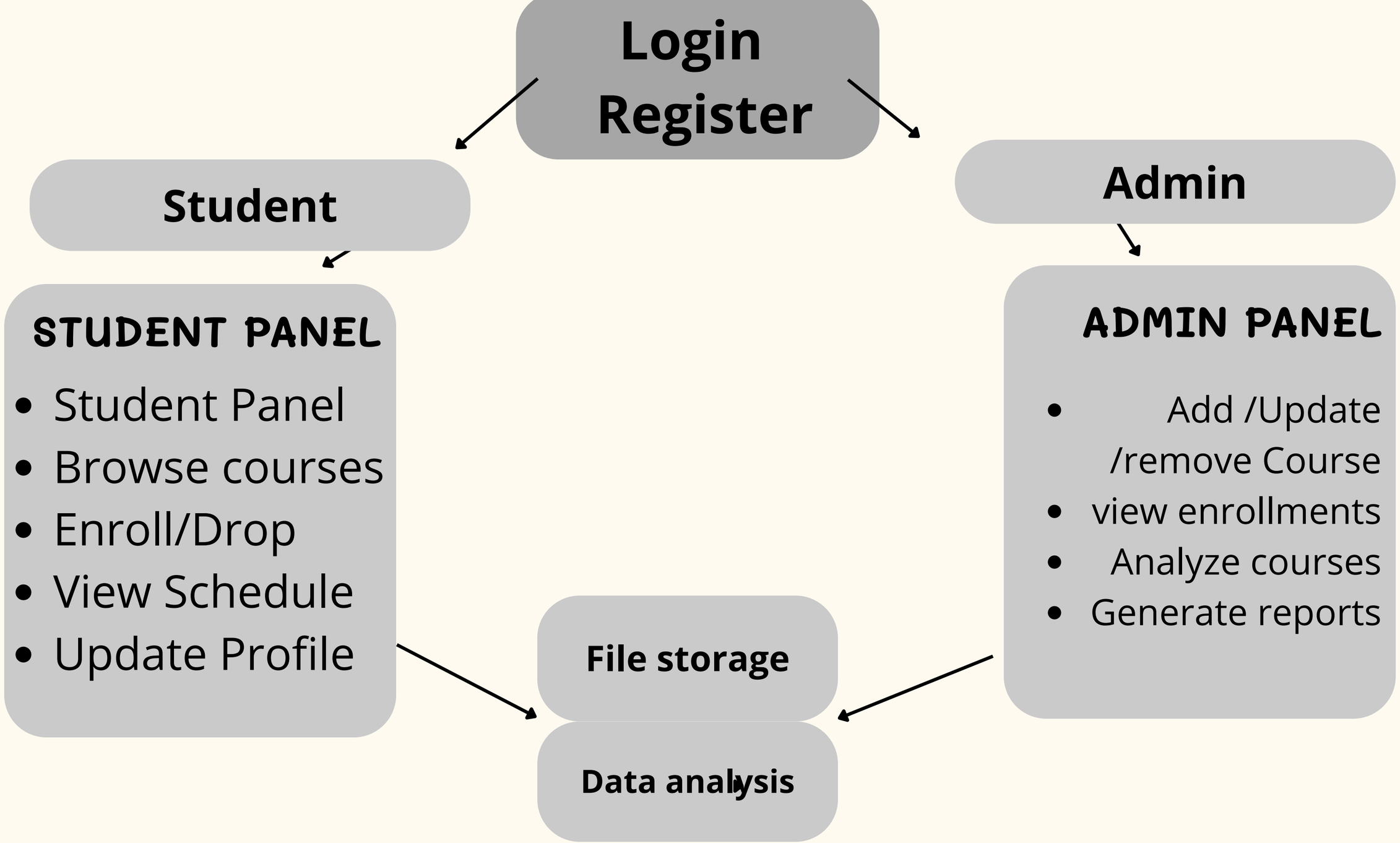


01. To create a system where the code is structured logically using specialized managers (Services) for users, courses, and enrollments. This makes the system easy to maintain and fix. We prove it works correctly by using unit tests to check every major function.



02. To provide all necessary features for both user types: Students can easily sign up, enroll in, and drop courses. Admins have the power to manage the course list and check student records. The system will strictly enforce rules, such as limiting the number of students in a class.




Login
Register
Student
Admin
STUDENT PANEL
Student Panel
Browse courses
Enroll/Drop
View Schedule
Update Profile
ADMIN PANEL
Add /Update /remove Course
view enrollments
Analyze courses
Generate reports
File storage
Data analysis

echnologies & Design Principles
Project approach

Technical Stack

Python 3
- Latest version for performance & async

Pytest
- Robuost, scalable testing framework



Design Principles

Object-Oriented Progamming (OOP): Encaplactions, reusablility

Modular Architecture (services, services, models, API)

Pytest for Testing

Soliabiity Maintenablle ✓ Code Quality

Slide 6: Project Structure

What Our Code Conists Of

Codebase Overview

- Modular Design
- Organized Folders/Files
- Focus on Readiability



Key Directories & Files

- “app/ (main application logic)
- tests/ (all unit/integration tests)
- “config.py (settings)
- “main.py (entry point)

File Organization Clean Code Modularity

🛠System Architecture

How our code is organized

Modules / packages

- models – represents data (Student, Course)
- services – core business logic (UserService, CourseService, EnrollmentService, AnalyticsService)
- utils – helper functions (hashing, validators)
- database – handles JSON read/write
- cli – command-line interface for users
- Persistence: JSON files (users.json, courses.json, enrollments.json)

(Optional diagram: boxes for modules and arrows showing interaction)

Student Tools & Administrator Control

🎓 Student Panel Features

Login/Register ✅

Browse Courses 🔎

Enroll/Drop Courses ➡️ ⬅️

View Schedule 🗓️

Update Profile

⚙️ Admin Panel Features

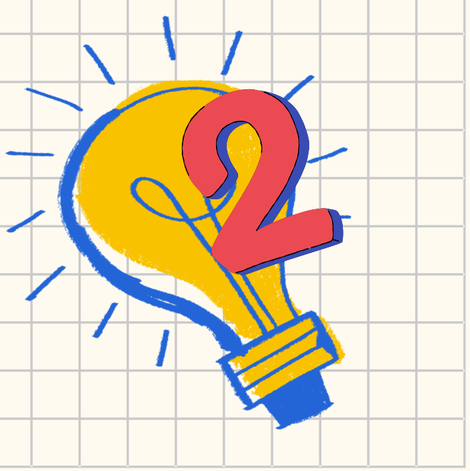
Login (Secure Access) 🔒

Course Management 📝

Add / Update / Remove Course

View Enrollments (Student Records) 📈

Generate Reports 📧







💾 Data Storage & Persistence

Uses JSON files in data/ directory:

users.json ------stores student & admin info 👨‍🎓

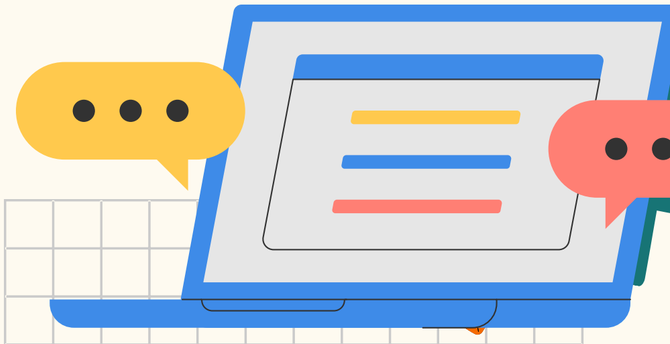
courses.json ------- stores course info 📚

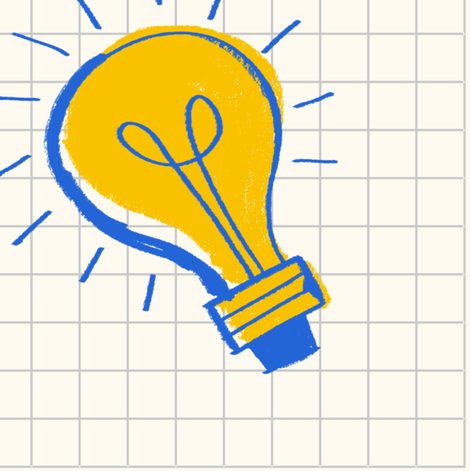
enrollments.json --------stores enrollment info ✅

Serialization: Converts objects to dict before saving. 📤💾

Deserialization: Loads dicts into objects on startup. 💾📥







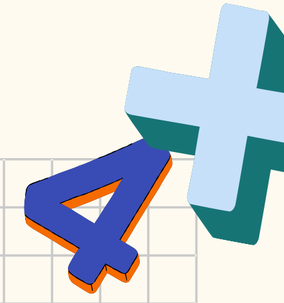




📝Enrollment Handling

Case-insensitive course ID matching. 🔍
Checks capacity before enrollment. 🚦
Stores username and timestamp in enrollment entry. 🕰️
Returns full schedule with enrolled_at for students. 🗓️





Backend Services for a Course Management Application ⚙️

User Management

StudentService 👨‍🎓

Register, authenticate, update profile

AdminService ⚙️

Default admin creation

Authentication for admin accounts

Analytics Features

Popular courses ranking 🔥

Enrollment trend over time 📈

Course capacity status 🚦

Individual student workload 🏋️

Course Management

CourseService 🏫

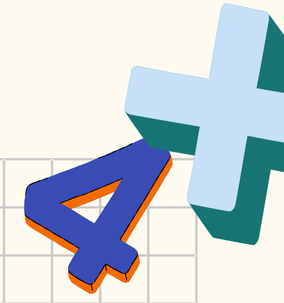
Add, update, remove courses

List courses

Sample courses creation

Case-insensitive retrieval 🔍





CLI Interface

💻Interactive menus for students and admins 🖱️

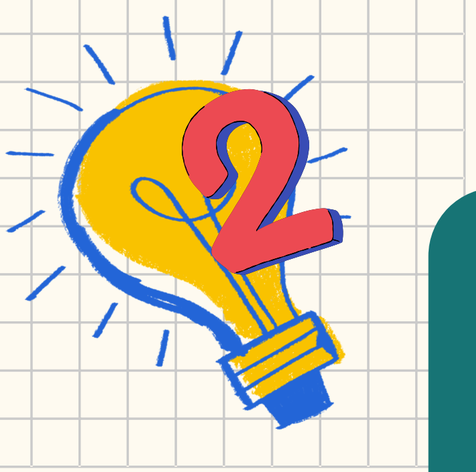
Clear screen & pause between actions ⏸️

Input validation:

Username: [A-Za-z0-9_-]{3,30}

Course ID: [A-Za-z0-9_-]{2,20}

Positive integers for credits/capacity 👍

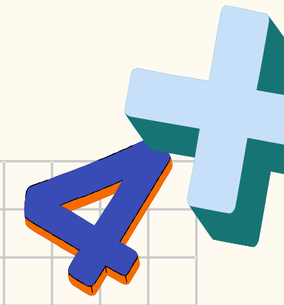




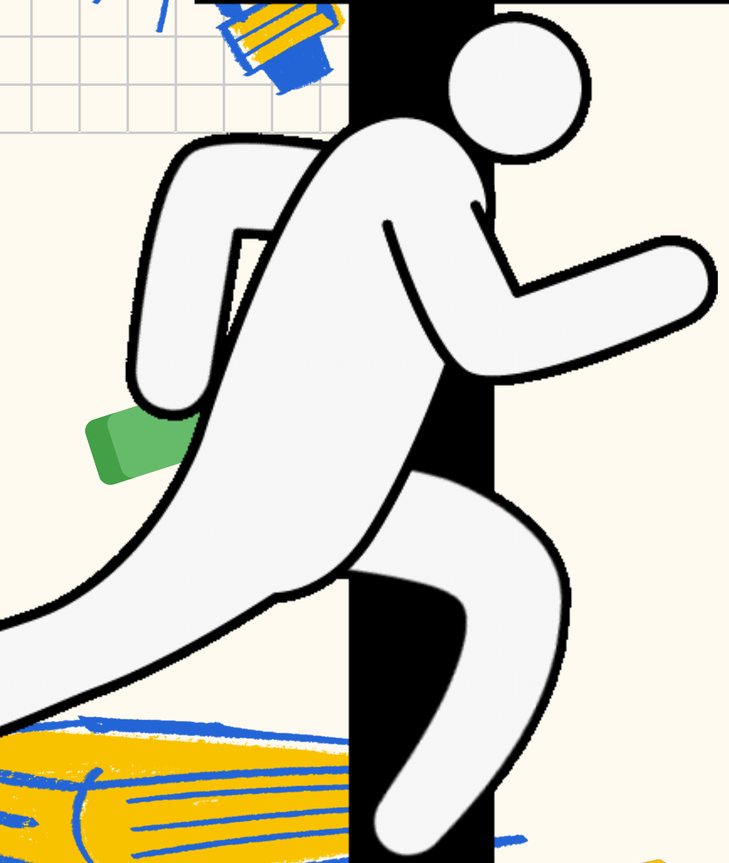
✅ Testing

- Uses pytest for unit testing
- Covers:
- User registration & authentication
- Course add/update/remove
- Enrollment and schedule
- Case-insensitive course handling
- Ensures JSON persistence works correctly





TIME TO RUN CODE




THANKYOU